

CO3 — ASSESSMENT TOOL 3

Resolution Algorithm Implementation

Artificial Intelligence (CSA1709)
Propositional & First-Order Logic, Unification,
Resolution-based Inference

Student Name	Avishek N
Register Number	192511009
Course Outcome	CO3
Assessment Weightage	20% Total Marks: 50

Prepared for Academic Submission

Declaration: I, Avishek N (Reg. No. 192511009), certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

CONTENTS & LEARNING OBJECTIVES

1. 1. Conjunctive Normal Form (CNF) Conversion
2. 2. Propositional Resolution (PL-Resolution)
3. 3. Unification Algorithm (UNIFY) and Most General Unifier (MGU)
4. 4. First-Order Resolution with Unification
5. 5. Forward and Backward Chaining
6. Overall conclusion and comparison

Assessment Objective

The purpose of this assessment is to demonstrate practical implementation of core symbolic-AI inference techniques. The solutions cover transformation of logical statements into CNF, refutation using propositional resolution, recursive unification with an occurs check, first-order resolution using substitutions, and rule-based inference through forward and backward chaining.

Method Used

- Represent logical knowledge in clauses and atomic predicates.
- Apply logically equivalent transformations before inference.
- Use explicit substitutions (θ) and Most General Unifiers (MGUs).
- Trace each inference step so that the final conclusion can be verified manually.
- Use Python implementations to automate and validate the reasoning process.

The question wording, examples, and terminology in this document are based on the uploaded CO3 assessment material.

filecite turn0file0 L31-L40

Question 1 — Conjunctive Normal Form (CNF) Conversion

Task: Convert the given propositional sentences to CNF by showing every transformation step and implement a program that outputs the CNF form.

1.1 Given Knowledge Base

- $(P \vee Q) \Rightarrow R$
- $R \Rightarrow \neg S$
- $S \vee T$
- $\neg T \Rightarrow P$

1.2 Manual CNF Conversion

Sentence 1: $(P \vee Q) \Rightarrow R$

- Eliminate implication: $\neg(P \vee Q) \vee R$.
- Apply De Morgan's law: $(\neg P \wedge \neg Q) \vee R$.
- Distribute OR over AND: $(\neg P \vee R) \wedge (\neg Q \vee R)$.
- Clauses: $\{\neg P, R\}$ and $\{\neg Q, R\}$.

Sentence 2: $R \Rightarrow \neg S$

- Eliminate implication: $\neg R \vee \neg S$.
- This is already in CNF.
- Clause: $\{\neg R, \neg S\}$.

Sentence 3: $S \vee T$

- No implication or conjunction is present.
- Therefore it is already in CNF.
- Clause: $\{S, T\}$.

Sentence 4: $\neg T \Rightarrow P$

- Eliminate implication: $\neg(\neg T) \vee P$.

- Double negation gives $T \vee P$.
- Clause: $\{T, P\}$.

1.3 Final CNF

$(\neg P \vee R) \wedge (\neg Q \vee R) \wedge (\neg R \vee \neg S) \wedge (S \vee T) \wedge (T \vee P)$

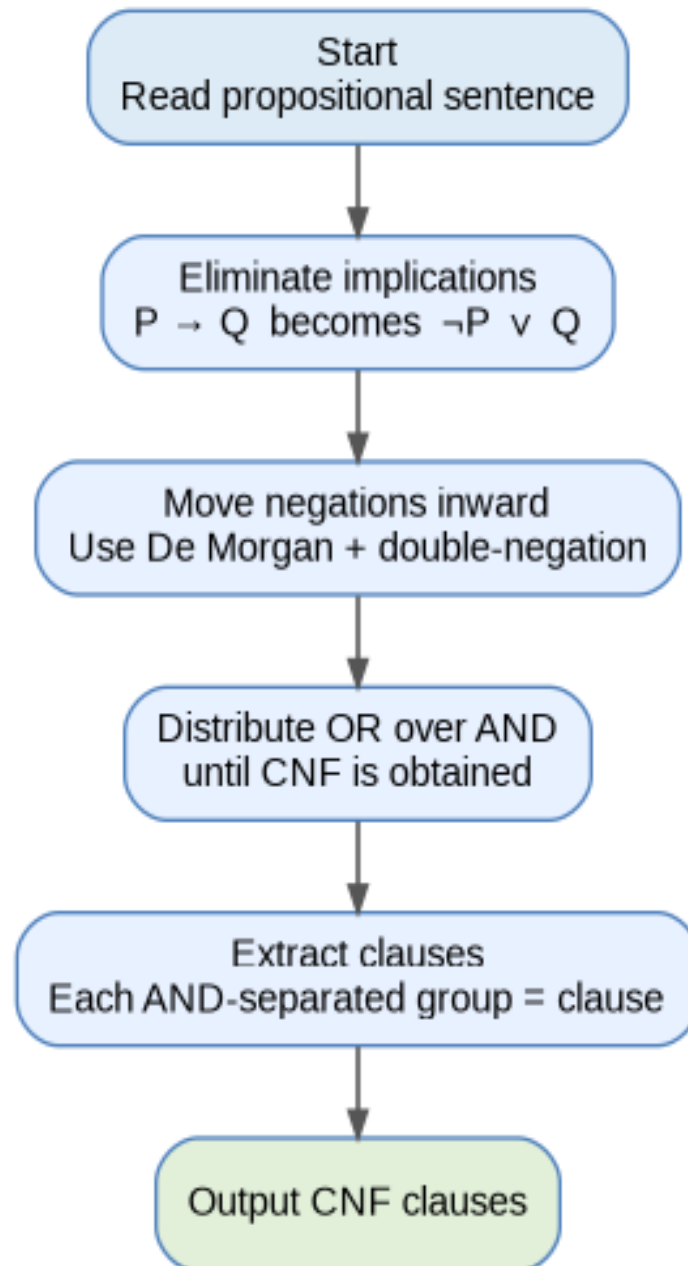


Figure 1. Mermaid-style CNF conversion flow

Question 1 — Program Implementation

A symbolic implementation can use SymPy's Boolean logic utilities. The program performs implication elimination and converts each expression to an equivalent CNF representation.

```
from sympy import symbols
from sympy.logic.boolalg import to_cnf, Implies, Or, Not

P, Q, R, S, T = symbols('P Q R S T')

sentences = [
    Implies(Or(P, Q), R),
    Implies(R, Not(S)),
    Or(S, T),
    Implies(Not(T), P)
]

for expr in sentences:
    print("Original :", expr)
    print("CNF      :", to_cnf(expr, simplify=True))
    print("-" * 40)
```

Expected Output

- $\text{Implies}(P \mid Q, R) \rightarrow (R \mid \sim P) \ \& \ (R \mid \sim Q)$
- $\text{Implies}(R, \sim S) \rightarrow \sim R \mid \sim S$
- $S \mid T \rightarrow S \mid T$
- $\text{Implies}(\sim T, P) \rightarrow P \mid T$

The automated results match the manually derived clauses, confirming that the transformation is correct.

□ filecite□turn0file0□L57-L95□

Question 2 — Propositional Resolution

A Wumpus-World-style knowledge base is used to prove the query P12 by refutation. The method adds $\neg P12$ to the knowledge base and attempts to derive the empty clause.

- $\neg P11$
- $\neg B11 \vee P12 \vee P21$
- B11
- $\neg P21$
- Query: P12

The four clauses and the query are specified in the assessment material. □ filecite□turn0file0□L104-L117□

Question 2 — PL-Resolution

2.1 Refutation Setup

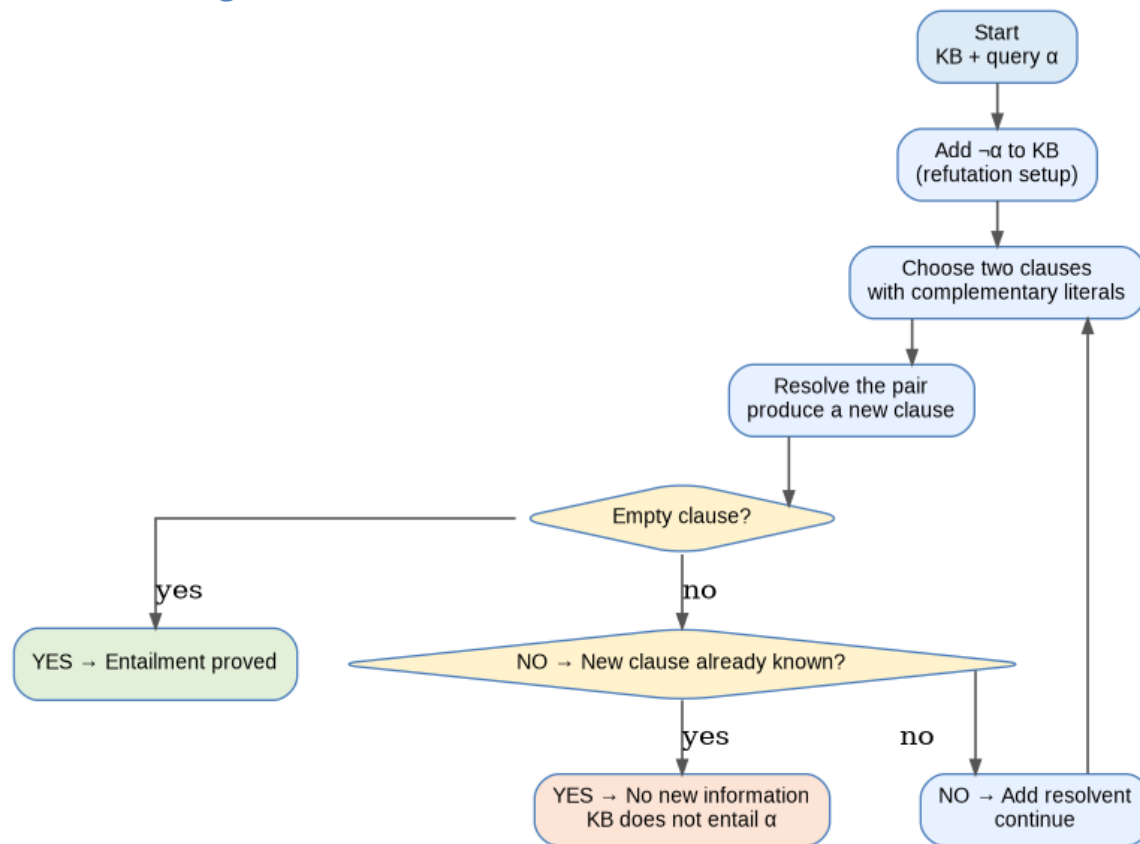
To test whether $KB \models P12$, negate the query and add $\neg P12$ to the clause set. Resolution repeatedly combines clauses containing complementary literals.

2.2 Minimal Refutation Chain

- R1: Resolve $(\neg B11 \vee P12 \vee P21)$ with (B11) $\rightarrow (P12 \vee P21)$.
- R2: Resolve $(P12 \vee P21)$ with $(\neg P21) \rightarrow (P12)$.
- R3: Resolve (P12) with $(\neg P12) \rightarrow \{\}$.

The symbol $\{\}$ is the empty clause and represents contradiction. Since $KB \wedge \neg P12$ produces a contradiction, the original query P12 is entailed.

2.3 PL-Resolution Algorithm



2.4 Result

Empty clause derived $\Rightarrow$ $KB \models P12$. Therefore, the agent can infer that square [1,2] contains a pit. The assessment's complete trace also reports the empty clause at the final resolution step. □ filecite□turn0file0□L195-L221□

Question 3 — Unification Algorithm (UNIFY) and MGU

Unification finds a substitution θ that makes two first-order terms identical. When the algorithm returns the least restrictive substitution capable of making the terms equal, it is called the Most General Unifier (MGU).

3.1 Recursive UNIFY Logic

- If the two terms are identical, return the current substitution.
- If the first term is a variable, use UNIFY-VAR.
- If the second term is a variable, use UNIFY-VAR symmetrically.
- If both are compound terms, their functors and arities must match; then unify corresponding arguments recursively.
- Reject a variable binding when the occurs check detects that the variable appears inside the term.

3.2 Test Pair Results

Pair	Atomic sentences	Result
(i)	Knows(John,x) and Knows(John,Jane)	{x/Jane}
(ii)	Knows(x,Elizabeth) and Knows(y,x)	{x/Elizabeth, y/Elizabeth}
(iii)	Knows(x,x) and Knows(y,Father(y))	FAILURE — occurs check

3.3 Occurs-Check Failure

For Pair (iii), the first arguments require x and y to be unified. The second arguments then require x to unify with $Father(y)$. After the first substitution, this becomes $y = Father(y)$. The variable y therefore occurs inside the term to which it would be bound. This would require an infinite self-referential term, which is not a finite first-order term. The occurs check correctly rejects the substitution.

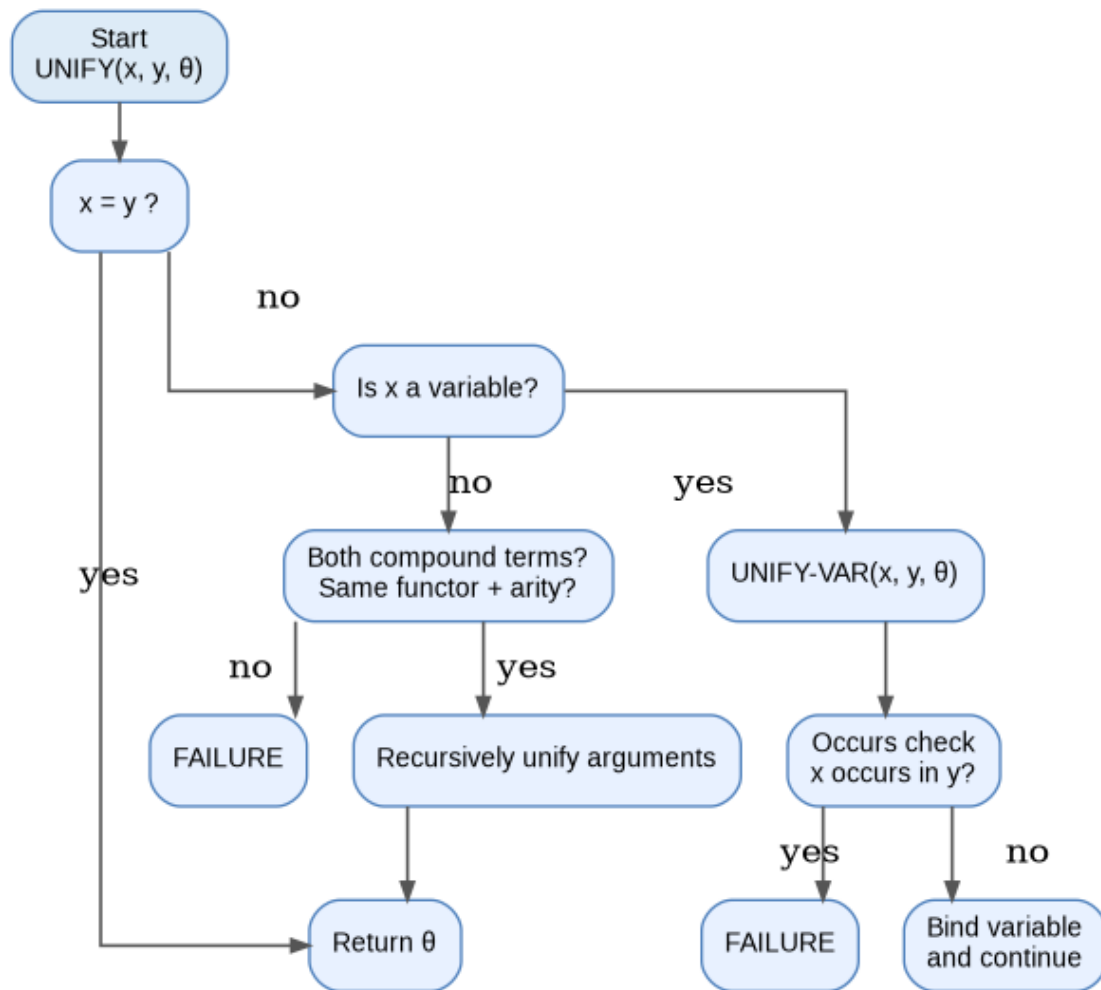


Figure 3. Mermaid-style UNIFY control flow

The three test results and the occurs-check explanation follow the uploaded assessment. ☐ filecite ☐ turn0file0 ☐ L322-L341 ☐

Question 4 — First-Order Resolution with Unification

First-order resolution extends propositional resolution by allowing variables and using a Most General Unifier before complementary literals are cancelled.

4.1 Family-Relationship Knowledge Base

- $\forall x,y,z: \text{Parent}(x,y) \wedge \text{Parent}(y,z) \Rightarrow \text{Grandparent}(x,z)$
- $\text{Parent}(\text{Tom}, \text{Bob})$
- $\text{Parent}(\text{Bob}, \text{Ann})$
- $\forall x,y: \text{Father}(x,y) \Rightarrow \text{Parent}(x,y)$
- $\text{Father}(\text{Tom}, \text{Bob})$
- Goal: $\text{Grandparent}(\text{Tom}, \text{Ann})$

4.2 CNF Clauses

- C1: $\neg \text{Parent}(x,y) \vee \neg \text{Parent}(y,z) \vee \text{Grandparent}(x,z)$
- C2: $\text{Parent}(\text{Tom}, \text{Bob})$
- C3: $\text{Parent}(\text{Bob}, \text{Ann})$
- C4: $\neg \text{Father}(x,y) \vee \text{Parent}(x,y)$
- C5: $\text{Father}(\text{Tom}, \text{Bob})$
- C6: $\neg \text{Grandparent}(\text{Tom}, \text{Ann})$ [negated goal]

4.3 Complete Refutation Proof

Step	Resolution	MGU θ	Resolvent
R1	C5 with C4	{x/Tom,y/Bob}	Parent(Tom,Bob)
R2	R1 with C1	{x/Tom,y/Bob}	$\neg$ Parent(Bob,z) $\vee$ Grandparent(Tom,z)
R3	R2 with C3	{z/Ann}	Grandparent(Tom,Ann)
R4	R3 with C6	{}	EMPTY CLAUSE

The empty clause at R4 proves the goal by refutation. The composed substitutions are {x/Tom, y/Bob, z/Ann}.

□ filecite□ turn0file0□ L350-L395□

Question 5 — Forward and Backward Chaining

A Horn-clause knowledge base is used to demonstrate two inference strategies for the query WantedByInterpol(West).

5.1 Knowledge Base

Facts

- F1: Owns(Nono,M1)
- F2: Missile(M1)
- F3: American(West)
- F4: Enemy(Nono,America)

Rules

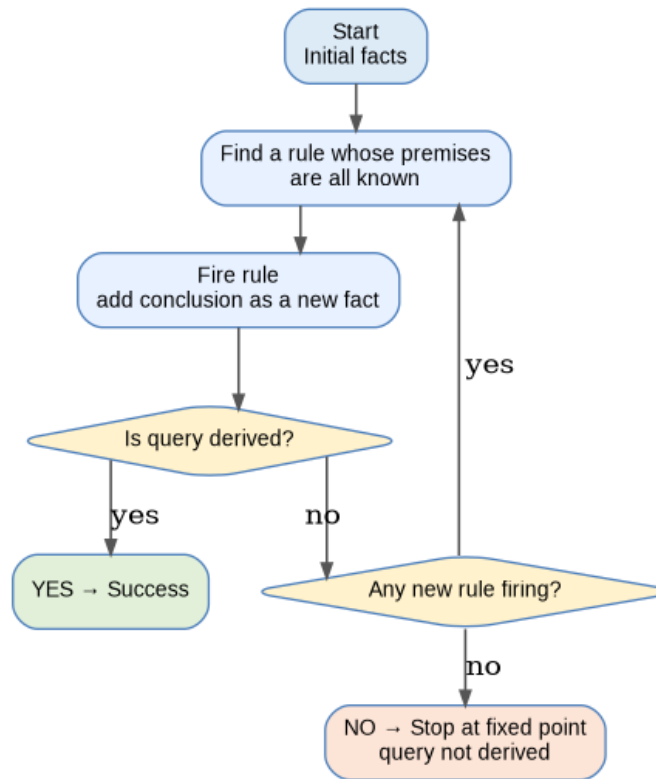
- R1: Missile(x) $\Rightarrow$ Weapon(x)
- R2: Owns(Nono,y) $\wedge$ Missile(y) $\Rightarrow$ Sells(West,y,Nono)
- R3: Enemy(x,America) $\Rightarrow$ Hostile(x)
- R4: American(x) $\wedge$ Weapon(y) $\wedge$ Sells(x,y,z) $\wedge$ Hostile(z) $\Rightarrow$ Criminal(x)
- R5: Criminal(x) $\Rightarrow$ WantedByInterpol(x)

5.2 Forward Chaining Trace

Forward chaining is data-driven. It starts from facts and repeatedly fires rules whose premises are satisfied.

Iteration	Rule	Substitution θ	New fact
1	R1	{x/M1}	Weapon(M1)
2	R2	{y/M1}	Sells(West,M1,Nono)
3	R3	{x/Nono}	Hostile(Nono)
4	R4	{x/West,y/M1,z/Nono}	Criminal(West)
5	R5	{x/West}	WantedByInterpol(West)

At iteration 5 the query is matched, so forward chaining succeeds. □ filecite□ turn0file0□ L459-L466□



Question 5 — Backward Chaining and Comparison

5.3 Backward-Chaining Goal Reduction

Backward chaining is goal-driven. It begins with the query, finds a rule whose conclusion matches the goal, and recursively proves the rule's premises.

- Goal WantedByInterpol(West) matches R5 → prove Criminal(West).
- Criminal(West) matches R4 → prove American(West), Weapon(y), Sells(West,y,z), and Hostile(z).
- American(West) is fact F3 → TRUE.
- Weapon(y) matches R1 with y=M1 → Missile(M1) is fact F2 → TRUE.
- Sells(West,M1,z) matches R2 → Owns(Nono,M1) and Missile(M1) are facts → Sells(West,M1,Nono) TRUE.
- Hostile(Nono) matches R3 → Enemy(Nono,America) is fact F4 → TRUE.
- All R4 subgoals are true → Criminal(West) TRUE → WantedByInterpol(West) PROVED.

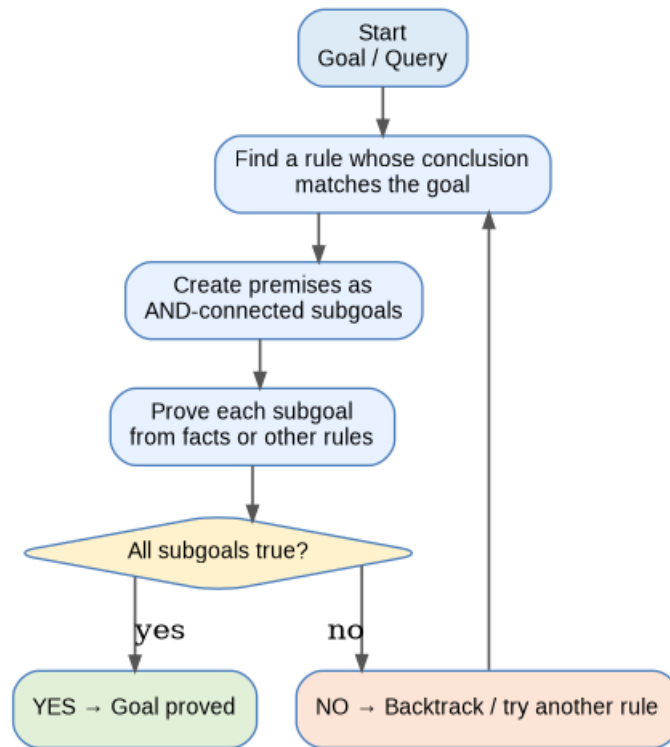


Figure 5. Mermaid-style Backward Chaining flow

5.4 Comparison

Aspect	Forward Chaining	Backward Chaining
Direction	Facts → rules → new facts → goal	Goal → rules → subgoals → facts
Search focus	May derive unrelated consequences	Focuses on the requested goal
Best suited	Continuous monitoring / expert systems	Query answering / diagnosis / theorem proving
Stopping	Fixed point or query match	All subgoals proved or branch fails
This example	More facts are derived before goal	More focused on the goal

For this knowledge base, backward chaining is more focused because it explores only the facts and rule instances needed to prove the requested query. □ filecite □ turn0file0 □ L475-L529 □

Overall Conclusion

This assessment demonstrates how symbolic reasoning can be implemented systematically from logical representation to inference. CNF conversion provides a standard clause representation that supports resolution. Propositional resolution proves entailment by contradiction, while first-order resolution extends the same idea through unification. The UNIFY algorithm supplies substitutions that make predicates compatible, and the occurs check prevents invalid self-referential bindings.

The final section demonstrates that rule-based systems can reason in two directions. Forward chaining begins with available facts and derives consequences, whereas backward chaining begins with a goal and reduces it to subgoals. In the given Horn-clause example, both methods prove `WantedByInterpol(West)`, but backward chaining is more focused for a single query.

Key Learning Outcomes

- Convert implications and negations into logically equivalent CNF clauses.
- Construct a refutation proof and identify the empty clause.
- Compute MGUs using recursive unification and apply the occurs check.
- Perform first-order resolution with substitutions at every step.
- Trace forward and backward chaining and explain their differences.
- Validate manual reasoning using small Python implementations.

Final Results at a Glance

Technique	Verified result
CNF Conversion	Five final clauses obtained from the four input sentences.
PL-Resolution	Empty clause derived; $KB \models P12$.
UNIFY	Two successful MGUs and one occurs-check failure.
FOL Resolution	Empty clause derived; $KB \models \text{Grandparent}(\text{Tom}, \text{Ann})$.
Forward Chaining	WantedByInterpol(West) derived at iteration 5.
Backward Chaining	WantedByInterpol(West) proved by goal reduction.

Code of Conduct:

I Avishek N (192511009) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Avishek N

RUBRICS FOR CALCULATION

Criteria	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Max Marks
CNF Conversion	Accurately converts all sentences to CNF, correctly applying implication elimination, negation (NNF) and distribution; program runs correctly on all test cases with clear, well-labelled output.	Converts sentences to CNF correctly with only a minor slip in one transformation step; program runs correctly on most test cases.	Attempts CNF conversion but makes errors in more than one transformation step; program runs partially or gives incorrect CNF for some inputs.	Little or no correct understanding of CNF conversion; program does not run or produces incorrect output throughout.	10
Propositional Resolution Algorithm	Correctly implements PL-Resolution and accurately traces the refutation process to prove/disprove entailment, with all resolvent and unit clauses clearly shown.	Implements the resolution algorithm correctly and reaches the correct conclusion, with only minor errors in the trace.	Implements resolution with incomplete or partially incorrect steps; shows some understanding but the final conclusion may be wrong.	Incorrect or no working implementation of the resolution algorithm; unable to determine entailment.	10
Unification Algorithm – MGU	Correctly implements the UNIFY algorithm and computes an accurate Most General Unifier for all test pairs,	Computes the MGU correctly for most pairs with only minor errors; failure case	Partial implementation of unification; produces an incorrect MGU for some pairs or does not	Little or no correct implementation of the unification algorithm.	10

	including correctly identifying the unification-failure case.	handled with a small issue.	correctly detect the failure case.		
First-Order Resolution Proof	Correctly converts the knowledge base and negated goal to CNF and constructs a complete, correct resolution-refutation proof establishing the goal.	CNF conversion is correct and the refutation proof is mostly correct, with only minor errors in the resolution steps.	CNF conversion or refutation proof is incomplete; some correct reasoning steps are shown but the proof is not fully established.	Incorrect or missing CNF conversion and resolution proof.	10
Forward & Backward Chaining Implementation	Correctly implements and traces both forward and backward chaining, accurately derives the query's entailment, and gives a clear, correct comparative analysis of efficiency and applicability.	Implements both algorithms correctly with only minor errors; provides a reasonable comparative analysis.	Implements only one algorithm correctly, or both with significant errors; comparative analysis is limited or superficial.	Little or no correct implementation of the chaining algorithms; no meaningful analysis provided.	10